A module may also be included at the top level, in which case the module's constants, class variables, and instance methods become available at the top level.

class|module *name*
  prepend *expr*
end

The method prepend behaves like include except that the prepended module's methods are placed in the lookup chain before the methods in the enclosing class or module.

class|module *name*
  extend *expr*
end

The method extend adds the modules methods directly to the singleton class of self. In typical usage, extend is at the top-level of a class, and so the new methods act as class or module methods of the enclosing class or module.

Module Functions

Instance methods defined in modules can be mixed-in to a class using include. But what if you want to call the instance methods in a module directly?

```
module Math
  def sin(x)
    #
  end
end
include Math      # The only way to access Math.sin
sin(1)
```

The method module_function solves this problem by taking module instance methods and copying their definitions into corresponding module methods.

```
module Math
  def sin(x)
    #
  end
  module_function :sin
end
Math.sin(1)
include Math
sin(1)
```

The instance method and module method are two different methods: the method definition is copied by module_function, not aliased.

You can also use module_function with no parameters, in which case all subsequent methods will be module methods.

Access Control

private ‹*symbol*›*
protected ‹*symbol*›*
public ‹*symbol*›*